BC01-FM04 · séance 3 · mardi 10 novembre 2026, matin

TP · La liste d'emprunts : le guide complet

Durée : la matinée. D'abord **Git et GitHub** (40 min), puis les sprints 1 et 2. Le sprint 3 peut se finir chez vous, à distance : votre code sera sur GitHub
Mode : **en binôme, chacun sur son ordinateur**, comme deux développeurs d'une même équipe : un dépôt GitHub commun, des tickets, une branche par ticket, une relecture avant chaque fusion
À rendre : **un zip par binôme**, `NOM1-NOM2-React-S3.zip`, le **vendredi 13 novembre à 18 h 00**, dans le Drive de la promo → **Rendus — à déposer ici** → **FM04 · Séance 3**. Le projet **sans `node_modules/`**
Fichiers : le **code base** est **dans le zip qui accompagne cette fiche** : l'application de la fin de la séance 2, avec son README, ses tickets et trois bouchons
Nouveau aujourd'hui : **Git** et **GitHub**, les outils avec lesquels on travaille à plusieurs sur le même code
C'est : l'exercice **Ex3** de la fiche du module (gestion d'état : store et actions), et une préparation de l'épreuve **E2** (C6 : état bien géré, pas de duplication)

Votre mission

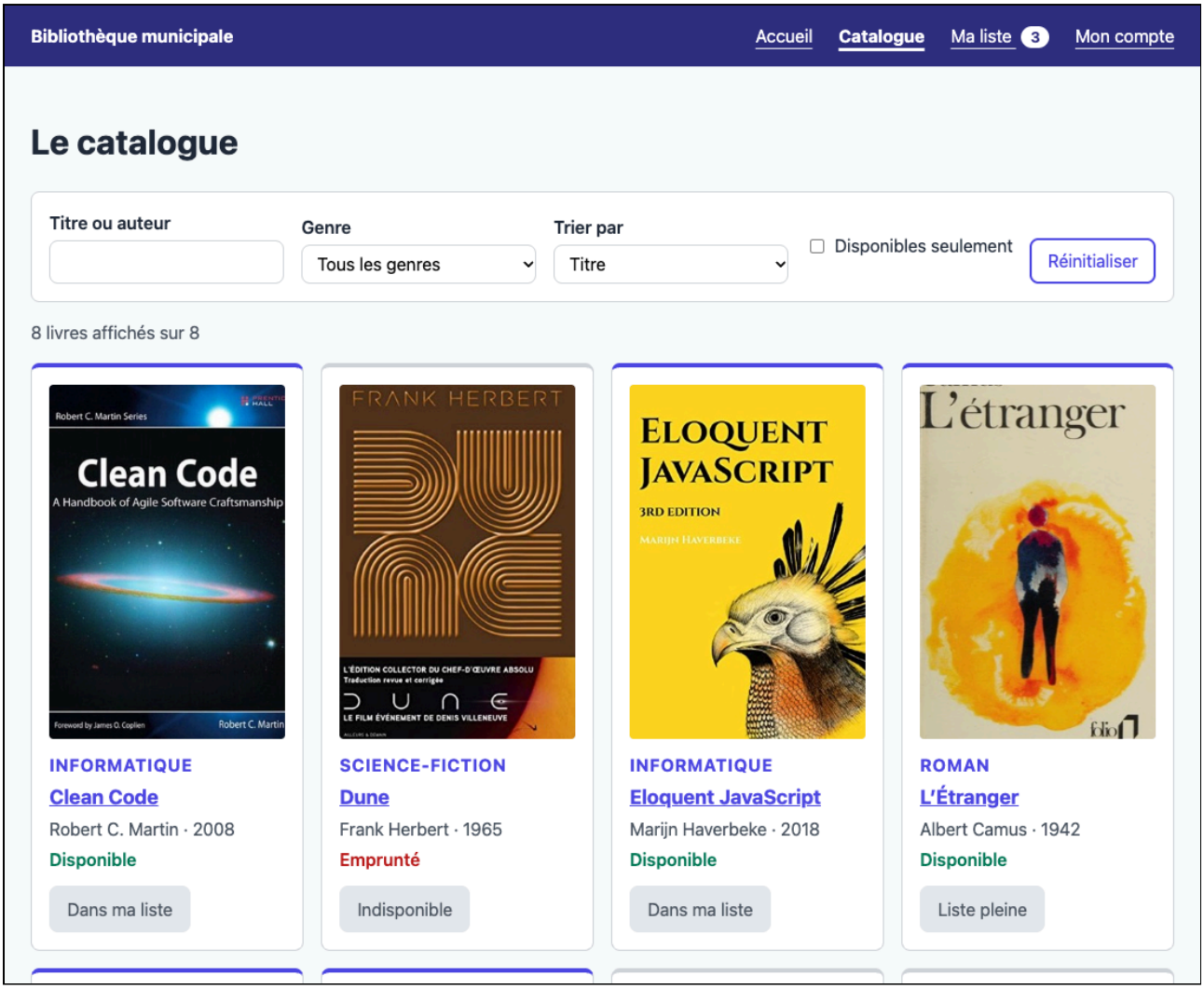
Vous êtes deux développeurs d'une agence web. Votre client, la bibliothèque municipale de Saint-Junien, a déjà son catalogue en ligne : c'est l'application de la séance 2. Il commande maintenant une **liste d'emprunts**.

Le lead technique de l'agence vous a préparé le terrain : un **code base propre**, un **README** qui dit comment l'équipe travaille, et le travail découpé en **sept tickets** (`TICKETS.md`). À vous deux de les livrer, comme dans une vraie équipe : chacun son ticket, chacun sa branche, et personne ne fusionne un code que l'autre n'a pas relu.

Ce que vous allez construire

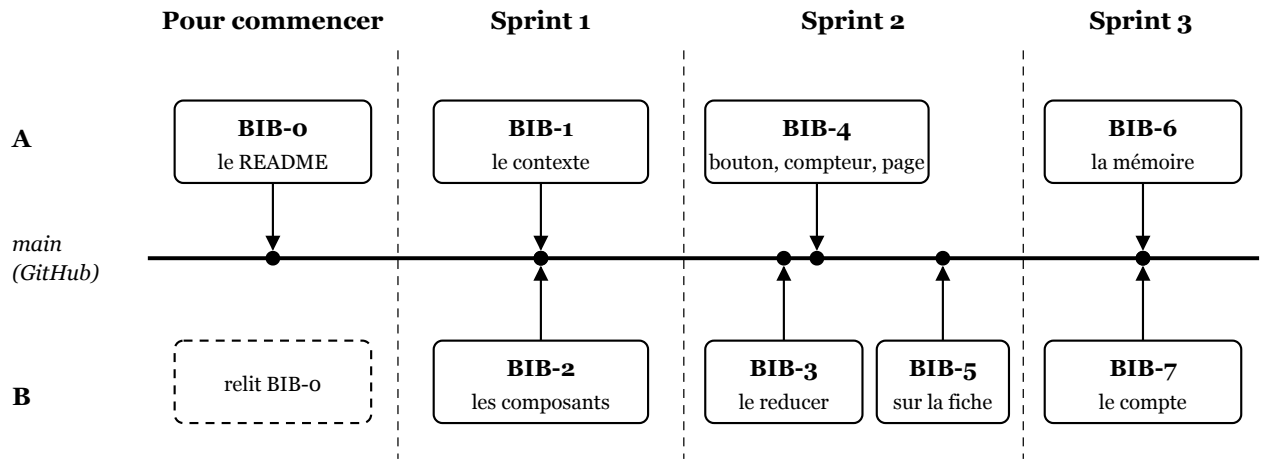
Une liste d'emprunts, comme un panier : un bouton « Ajouter à ma liste » sur chaque carte et sur chaque fiche, un compteur dans le menu, une page « Ma liste » pour retirer un livre ou tout vider. **Trois livres au plus**, chacun une fois. La liste **survit au rechargement** de la page.

Au passage, **le prop drilling de la séance 2 disparaît** : l'utilisateur connecté et la liste deviennent des valeurs que **tout composant** peut lire, sans les recevoir de son parent.



Le résultat attendu : trois livres dans la liste, le compteur du menu, et les boutons de chaque carte

Sprint	Ticket	Pour	Étapes	Ce que vous pratiquez
Jour 1	BIB-0 · Le README de l'équipe	A, relu par B		Git et GitHub : le dépôt, une branche, une pull request
1 · La connexion	BIB-1 · Le contexte de connexion	A	1, 2	`createContext`, un fournisseur, un hook `useAuth`
	BIB-2 · Les composants lisent la connexion	B	3	La fin du prop drilling
2 · La liste	BIB-3 · Le reducer et le contexte de la liste	B	4, 5	`useReducer`, des actions, un état qu'on ne modifie jamais
	BIB-4 · Le bouton, le compteur, la page	A	6, 7, 8	Un composant qui lit le contexte, réutilisé partout
	BIB-5 · Le bouton sur la fiche	B	9	Réutiliser le composant d'un collègue
3 · La mémoire	BIB-6 · La liste survit au rechargement	A	10	`localStorage`, `useEffect`, l'initialisation paresseuse
	BIB-7 · Le compte affiche la liste	B	11	Lire deux contextes



Qui fait quoi, et quand. Chaque ticket rejoint main par une pull request, relue par l'autre.

Les rôles s'inversent d'un sprint à l'autre : celui qui écrit le contexte au sprint 1 écrit les composants au sprint 2, et l'inverse. À la fin, chacun a fait les deux métiers.

Comment on travaille dans cette équipe

Formez un binôme, et décidez qui est A et qui est B. Chacun travaille **sur son propre ordinateur**. Personne ne regarde l'autre taper : vous écrivez tous les deux du code, **en même temps**, dans des fichiers différents. C'est Git qui réunit votre travail.

Chaque sprint se passe en quatre temps :

1. **Le contrat** (2 minutes, ensemble). Vous lisez le tableau « Le contrat » : c'est ce que le code de l'un promet au code de l'autre. Les noms qu'il contient ne se discutent pas : si l'un écrit `connecter` et l'autre `connexion`, rien ne s'emboîtera.
2. **Chacun son ticket, sur sa branche.** L'un écrit ce qui **fournit** une valeur, l'autre les composants qui la **lisent**. Celui qui lit n'a pas encore le vrai fichier de son collègue : le lead technique a posé à sa place un **bouchon**, un faux fichier de quelques lignes qui respecte le contrat. C'est ce qui permet de travailler à deux sans s'attendre.
3. **La livraison** : chacun envoie sa branche sur GitHub et ouvre une **pull request**. L'autre la **relit**, l'auteur explique son code, puis on **fusionne**.
4. **La recette** (ensemble) : vous récupérez tous les deux le code réuni, et vous vérifiez que tout marche.

Deux documents. La **fiche A** et la **fiche B** sont courtes : elles disent **quoi faire**, ticket par ticket, sans explication. Ce **guide complet** explique **pourquoi** : gardez-le à côté, et venez-y quand un morceau de code ou une commande vous échappe. Si vous travaillez seul, c'est lui que vous suivez.

Vous êtes seul ? Vous êtes A et B à la fois : faites les tickets dans l'ordre, chacun sur sa branche, et fusionnez vous-même vos pull requests.

Ce que vous tapez est expliqué. L'encadré **« Le JavaScript derrière »** explique le code ; l'encadré **« Git derrière »** explique les commandes. Lisez-les : à la relecture, c'est avec eux que vous expliquez votre travail à votre collègue.

Gardez les outils de React ouverts (onglet *Components*) : c'est là qu'on voit les contextes et l'état de la liste.

Jour 1 · Votre poste et le dépôt · 40 min

1 · Vérifier son poste · chacun · 10 min

Toutes les commandes Git se tapent dans le terminal de VS Code (*Terminal → Nouveau terminal*) : c'est lui qui saura vous connecter à GitHub.

```
git --version
```

Un numéro de version s'affiche : Git est là. Sinon, installez-le : sous Windows, **git-scm.com** (gardez tous les réglages proposés, puis relancez VS Code) ; sous macOS, la commande `xcode-select --install`.

Un compte GitHub. Si vous n'en avez pas, créez-le sur **github.com** (*Sign up*). Donnez votre identifiant à votre binôme.

Dites à Git qui vous êtes, une fois pour toutes, avec votre nom et l'adresse e-mail de votre compte GitHub :

```
git config --global user.name "Prénom Nom"
git config --global user.email "vous@exemple.fr"
```

> **Git derrière**
>
> - **Git** est un programme, sur votre ordinateur : il garde l'**historique** de votre projet, une photo à chaque étape. **GitHub** est un site : il héberge une copie de cet historique, que toute l'équipe partage.
> - `git config --global` écrit un réglage pour tous vos projets. Chaque photo de l'historique sera signée de ce nom.

---

2 · A crée le dépôt · 10 min

Pendant ce temps, B ouvre le zip pour lire `README.md` et `TICKETS.md` : ce sont les règles de l'équipe et le travail à faire. B n'installe rien : il recevra le projet par Git.

A : ouvrez le zip « code de départ », placez le dossier `catalogue-react/` dans votre dossier de cours, et ouvrez-le dans VS Code. Dans le terminal :

```
git init
git add .
git commit -m "Code de départ"
git branch -M main
```

> **Git derrière**
>
> - `git init` fait du dossier un **dépôt** : Git commence à le surveiller.
> - `git add .` choisit ce qui entrera dans la prochaine photo : ici, tout (le point veut dire « ce dossier »).
> - `git commit -m "…"` prend la photo, avec un message qui dit ce qu'elle contient. Une photo s'appelle un **commit**.
> - `git branch -M main` nomme la branche principale `main`.
> - Le fichier `.gitignore`, fourni, liste ce que Git ne doit **jamais** enregistrer : `node_modules` et `dist`. Ils se refabriquent avec `npm install` : on ne les partage pas.

Sur GitHub : le bouton +, en haut à droite, puis *New repository*. Nom : `catalogue-react`. Choisissez *Public*. **Ne cochez rien d'autre** (ni README, ni .gitignore : vous les avez déjà). *Create repository*.

GitHub affiche l'adresse du dépôt. Reliez-y votre dossier, et envoyez votre code :

```
git remote add origin https://github.com/VOTRE-IDENTIFIANT/catalogue-react.git
git push -u origin main
```

Au premier envoi, une fenêtre demande de vous connecter à GitHub : acceptez, et suivez-la dans le navigateur.

> **Git derrière**
>
> - `git remote add origin …` donne à Git l'adresse du dépôt en ligne, sous le nom `origin`.
> - `git push` **envoie** vos commits sur GitHub. `-u origin main` retient la destination : la prochaine fois, `git push` suffira.

Vérification : rechargez la page du dépôt sur GitHub. Vos fichiers y sont, et le README s'affiche en dessous.

3 · A invite B, B récupère le projet · 10 min

A, sur la page du dépôt : *Settings → Collaborators → Add people*, et l'identifiant de B.

B : acceptez l'invitation (elle arrive par e-mail, et dans les notifications de GitHub). Puis, dans le terminal de VS Code, placé dans votre dossier de cours :

```
git clone https://github.com/IDENTIFIANT-DE-A/catalogue-react.git
```

Ouvrez le dossier `catalogue-react/` ainsi créé dans VS Code.

Tous les deux, dans le terminal du projet :

```
npm install
npm run dev
```

> **Git derrière**
>
> - `git clone` **copie** un dépôt de GitHub sur votre ordinateur, avec tout son historique. B a maintenant exactement le projet de A.
> - `node_modules` n'a pas voyagé : il est dans `.gitignore`. D'où le `npm install` de chacun.

Vérification : le catalogue s'affiche chez vous deux, à l'adresse http://localhost:5173.

---

4 · La boucle d'un ticket, sur un exemple : BIB-0 · 10 min

Votre premier ticket ne contient aucun code : il sert à faire **une fois** tout le trajet. **A écrit, B relit.**

A crée une branche pour le ticket :

```
git switch -c bib-0-readme
```

A ouvre `README.md`, et complète deux lignes : les noms de l'équipe, et l'adresse du dépôt. Puis :

```
git status
git add .
git commit -m "BIB-0 : l'équipe et l'adresse du dépôt"
git push -u origin bib-0-readme
```

A, sur GitHub : un bandeau jaune propose *Compare & pull request*. Cliquez, vérifiez le titre, puis *Create pull request*.

B, sur GitHub : onglet *Pull requests*, ouvrez celle de A. Onglet *Files changed* : en rouge ce qui a été retiré, en vert ce qui a été ajouté. Si tout est bon : *Review changes → Approve → Submit review*.

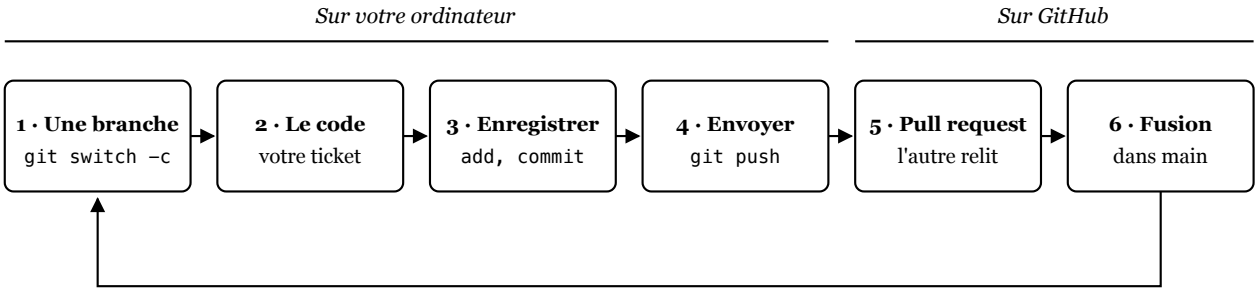
A : *Merge pull request → Confirm merge*, puis *Delete branch*.

Tous les deux se remettent à jour :

```
git switch main
git pull
```

> **Git derrière**
>
> - Une **branche** est une ligne de travail à part : ce que vous y faites ne touche pas `main` tant que ce n'est pas fusionné. `git switch -c nom` la crée et vous y place ; `git switch main` vous ramène sur `main`.
> - `git status` dit où vous en êtes : la branche, et les fichiers modifiés. À taper avant chaque `git add`.
> - Une **pull request** est une demande : « voici ma branche, relisez-la, puis fusionnez-la dans `main` ». C'est là que l'équipe relit le code.
> - `git pull` **récupère** ce qui a changé sur GitHub. Après chaque fusion, tout le monde le fait.

Vérification : chez B, `README.md` contient les noms de l'équipe, alors que B ne l'a jamais modifié.



Tous les deux : git switch main, git pull, puis le ticket suivant

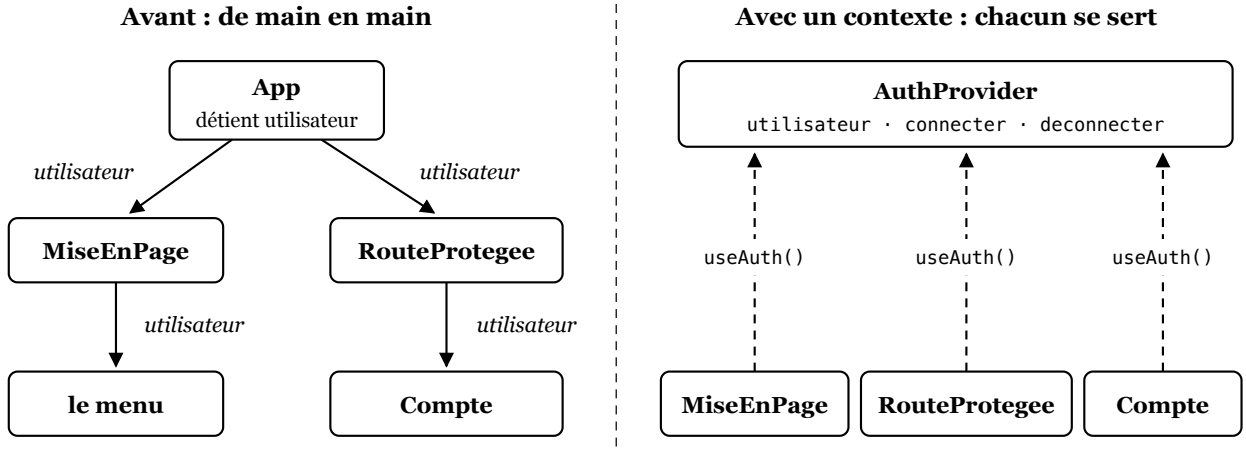
La boucle d'un ticket : trois gestes sur votre ordinateur, trois sur GitHub

C'est la boucle de tous vos tickets. Vous la referez sept fois :

Moment	**Commandes, ou bouton sur GitHub**
Avant de commencer un ticket	`git switch main`, `git pull`, puis `git switch -c bib-N-nom`
Le ticket est fini	`git status`, `git add .`, `git commit -m "BIB-N : …"`, `git push -u origin bib-N-nom`
Demander la relecture	*Compare & pull request*, puis *Create pull request*
Relire le ticket de l'autre	*Files changed*, puis *Review changes → Approve*
Fusionner	*Merge pull request → Confirm merge → Delete branch*
Se remettre à jour	`git switch main`, `git pull`

Partie 1 · Sprint 1 · Un contexte : qui est connecté

BIB-1, pour A : écrire le contexte (étapes 1 et 2). **BIB-2, pour B** : les composants qui le lisent (étape 3). Vous travaillez en même temps.



Ce que change le sprint 1 : l'utilisateur ne descend plus de main en main

Le contrat du sprint 1, à lire ensemble avant de commencer :

Le fichier `src/contexts/AuthContext.jsx` fournit	**Ce que c'est**
`AuthProvider`	Le composant à placer autour de l'application
`useAuth()`	Une fonction qui renvoie un objet avec trois choses :
→ `utilisateur`	`null` si personne n'est connecté, sinon un objet `{ id, nom, email }`
→ `connecter(adherent)`	Une fonction : connecte cet adhérent
→ `deconnecter()`	Une fonction : déconnecte

Ce fichier existe déjà : c'est un bouchon. Ouvrez-le ensemble. Il tient le contrat, mais personne n'y est jamais connecté. A va le remplacer par le vrai ; B peut déjà s'en servir.

Chacun crée sa branche, à partir d'un `main` à jour. A :

```
git switch main
git pull
git switch -c bib-1-contexte-connexion
```

Et B :

```
git switch main
git pull
git switch -c bib-2-lire-la-connexion
```

Étape 1 · A · BIB-1 · Créer le contexte de connexion · 15 min

À la séance 2, `utilisateur` vivait dans `App`, et descendait par les props vers quatre composants. Un **contexte** le met à disposition de **tous** les composants placés dessous, sans passer par chaque niveau.

Remplacez tout le contenu de `src/contexts/AuthContext.jsx` par :

```
import { createContext, useContext, useState } from 'react';

// Qui est connecté : une valeur que toute l'application peut lire, sans la passer
de main en main.
const AuthContext = createContext(null);

export function AuthProvider({ children }) {
  const [utilisateur, setUtilisateur] = useState(null);

  const valeur = {
    utilisateur,
    connecter: setUtilisateur,
    deconnecter: () => setUtilisateur(null),
  };

  return <AuthContext value={valeur}>{children}</AuthContext>;
}

export function useAuth() {
  const contexte = useContext(AuthContext);
  if (!contexte) {
    throw new Error('useAuth doit être utilisé dans un <AuthProvider>.');
  }
  return contexte;
}
```

Trois morceaux, toujours les mêmes :

Morceau	Son rôle
`createContext(null)`	Crée le « canal ». `null` : ce qu'on lit si aucun fournisseur n'est au-dessus
`AuthProvider`	Un composant **fournisseur** : il détient l'état, et le **diffuse** à tout ce qu'il enveloppe (`children`) avec `<AuthContext value={…}>`
`useAuth()`	Un **hook à nous** : il lit le contexte, et signale clairement l'oubli du fournisseur

Un hook, c'est une fonction dont le nom commence par `use`, qui appelle d'autres hooks. `useAuth` cache `useContext(AuthContext)` : les composants n'ont pas besoin de savoir comment c'est rangé.

> **Le JavaScript derrière**
>
> - `export function AuthProvider…` : le fichier partage **deux** fonctions par leur nom, sans `default`. Un autre fichier les importe entre accolades : `import { AuthProvider } from …`.
> - `{ children }` : une prop spéciale, ce que le composant **enveloppe** dans le JSX. Ici, ce sera toute l'application.
> - `const valeur = { utilisateur, connecter: setUtilisateur, … }` : un **objet**. Écrire `utilisateur` seul est un raccourci pour `utilisateur: utilisateur`. Une propriété peut contenir une fonction : `connecter` **est** `setUtilisateur`, sous un autre nom.
> - `() => setUtilisateur(null)` : une fonction fléchée **sans paramètre**, qui remet l'utilisateur à `null` quand on l'appelle.
> - `throw new Error('…')` : **lever une erreur** arrête la fonction et affiche le message en rouge dans la console. Ici, elle prévient celui qui a oublié le fournisseur.

Vérification : rien ne change encore à l'écran.

Étape 2 · A · BIB-1 · Le fournir à toute l'application · 5 min

Dans `src/main.jsx`, juste après la ligne `import { BrowserRouter } from 'react-router';` :

```
import { AuthProvider } from './contexts/AuthContext.jsx';
```

Dans `src/main.jsx`, remplacez :

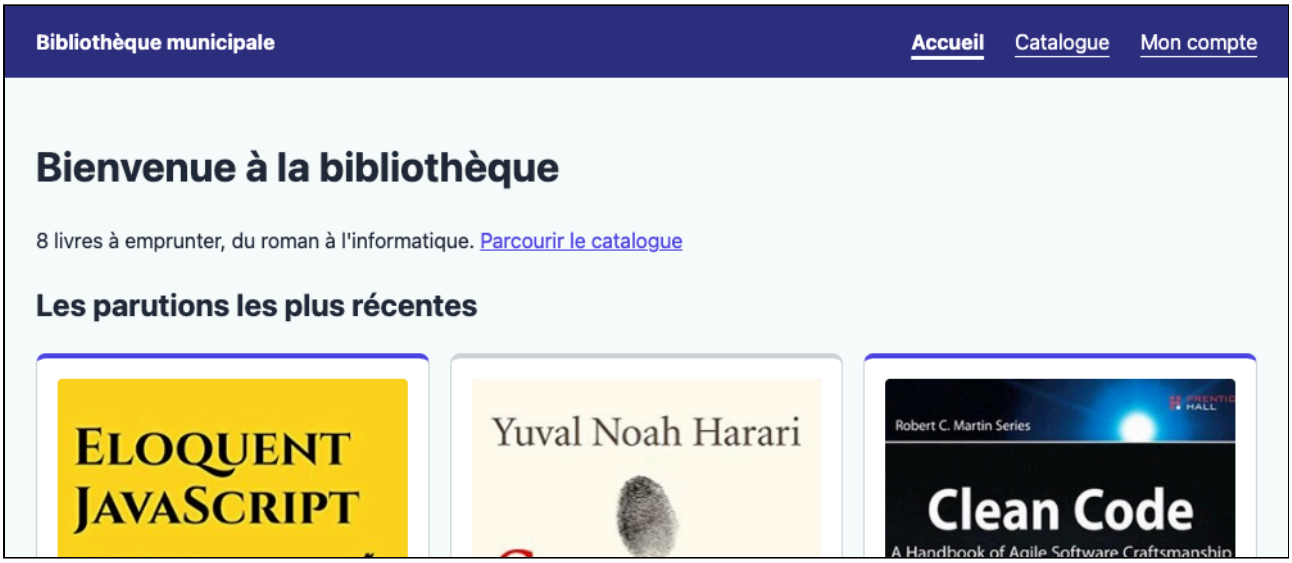
```
      <App />
```

par :

```
<AuthProvider>
  <App />
</AuthProvider>
```

> **Le JavaScript derrière**
>
> - Ce qui est écrit **entre** `<AuthProvider>` et `</AuthProvider>` devient sa prop `children`. `AuthProvider` reçoit donc toute l'application, et l'affiche telle quelle, avec le contexte autour.
> - L'ordre compte : seuls les composants placés **sous** le fournisseur peuvent lire le contexte.

Vérification : dans l'onglet *Components*, un `AuthProvider` apparaît au-dessus d'`App`. Cliquez dessus : son état, `null`.



BIB-1 : à l'écran, l'application s'affiche comme avant

Étape 3 · B · BIB-2 · Le lire là où on en a besoin · 20 min

B, vous commencez en même temps que A, sur votre branche. Vous utilisez le bouchon : avec lui, personne n'est jamais connecté, et c'est normal. Il vous permet d'écrire et de faire tourner votre code pendant que A écrit le vrai contexte, sur la sienne.

`App` n'a plus d'état à détenir, ni de props à distribuer.

Remplacez tout le contenu de `src/App.jsx` par :

```
import { Routes, Route } from 'react-router';
import MiseEnPage from './components/MiseEnPage.jsx';
import RouteProtegee from './components/RouteProtegee.jsx';
import Accueil from './pages/Accueil.jsx';
import Catalogue from './pages/Catalogue.jsx';
import FicheLivre from './pages/FicheLivre.jsx';
import Connexion from './pages/Connexion.jsx';
import Compte from './pages/Compte.jsx';
import PageIntrouvable from './pages/PageIntrouvable.jsx';

// Les adresses de l'application : une route = une adresse = une page.
export default function App() {
  return (
    <Routes>
      <Route element={<MiseEnPage />}>
        <Route index element={<Accueil />} />
        <Route path="catalogue" element={<Catalogue />} />
        <Route path="livres/:id" element={<FicheLivre />} />
        <Route path="connexion" element={<Connexion />} />
        <Route
          path="compte"
          element={
            <RouteProtegee>
              <Compte />
            </RouteProtegee>
          }
        />
        <Route path="*" element={<PageIntrouvable />} />
      </Route>
    </Routes>
  );
}
```

Chaque composant qui en a besoin **va chercher** l'utilisateur lui-même.

La mise en page.

Dans `src/components/MiseEnPage.jsx`, juste après la ligne `import { Link, NavLink, Outlet } from 'react-router';` :

```
import { useAuth } from '../contexts/AuthContext.jsx';
```

Dans `src/components/MiseEnPage.jsx`, remplacez :

```
export default function MiseEnPage({ utilisateur, onDeconnexion }) {
```

par :

```
export default function MiseEnPage() {
  const { utilisateur, deconnecter } = useAuth();
```

Dans `src/components/MiseEnPage.jsx`, remplacez :

```
                <button type="button" onClick={onDeconnexion}>Se
déconnecter</button>
```

par :

```
                <button type="button" onClick={deconnecter}>Se déconnecter</button>
```

La garde.

Dans `src/components/RouteProtegee.jsx`, juste après la ligne `import { Navigate, useLocation } from 'react-router';` :

```
import { useAuth } from '../contexts/AuthContext.jsx';
```

Dans `src/components/RouteProtegee.jsx`, remplacez :

```
export default function RouteProtegee({ utilisateur, children }) {
```

par :

```
export default function RouteProtegee({ children }) {
  const { utilisateur } = useAuth();
```

La connexion.

Dans `src/pages/Connexion.jsx`, juste après la ligne `import { adherents } from '../data/adherents.js';` :

```
import { useAuth } from '../contexts/AuthContext.jsx';
```

Dans `src/pages/Connexion.jsx`, remplacez :

```
export default function Connexion({ onConnexion }) {
```

par :

```
export default function Connexion() {
  const { connecter } = useAuth();
```

Dans `src/pages/Connexion.jsx`, remplacez :

```
    onConnexion({ id: adherent.id, nom: adherent.nom, email: adherent.email });
```

par :

```
    connecter({ id: adherent.id, nom: adherent.nom, email: adherent.email });
```

Le compte.

Remplacez tout le contenu de `src/pages/Compte.jsx` par :

```
import { useAuth } from '../contexts/AuthContext.jsx';

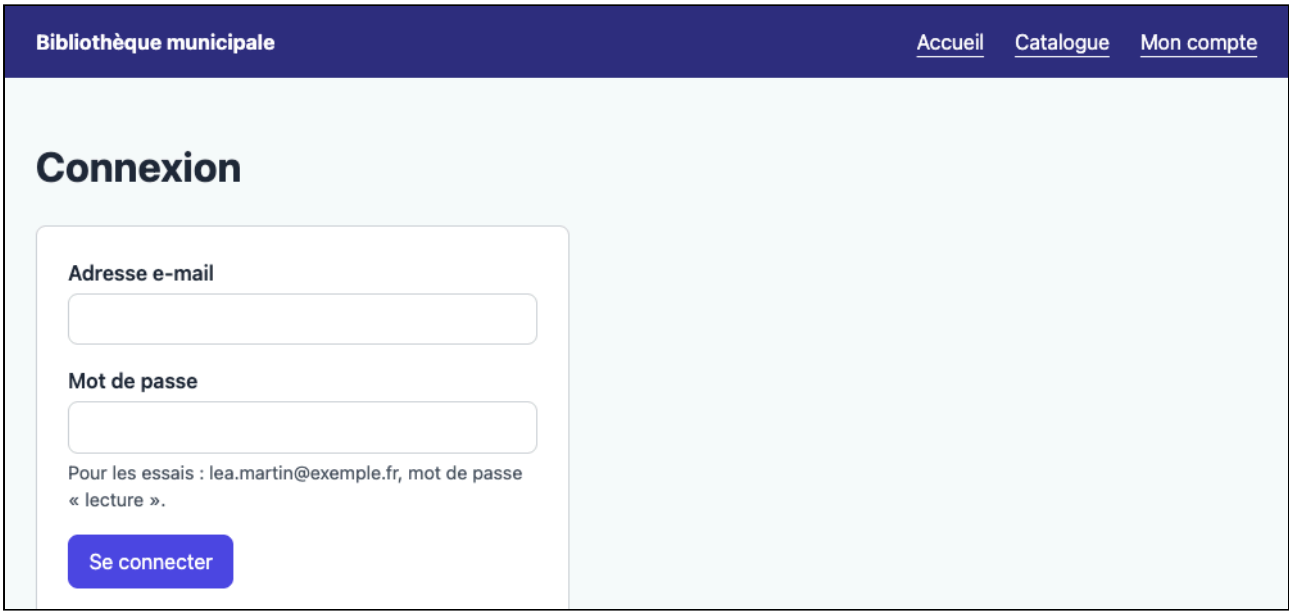
export default function Compte() {
  const { utilisateur } = useAuth();

  return (
    <>
      <title>Mon compte — Bibliothèque municipale</title>
      <h1>Mon compte</h1>
      <p>Bonjour {utilisateur.nom}. Adresse de connexion : {utilisateur.email}.</p>
      <p>Votre liste d'emprunts apparaîtra ici.</p>
    </>
  );
}
```

Comparez `App` avec la fin de la séance 2 : plus d'état, plus une seule prop. Il ne fait plus que ce que dit son commentaire : la liste des adresses.

> **Le JavaScript derrière**
>
> - `const { utilisateur, deconnecter } = useAuth();` : `useAuth()` renvoie l'objet `valeur` de l'étape 1, et on en sort directement les deux morceaux utiles. C'est la même **déstructuration** que pour les props, mais sur le résultat d'une fonction.
> - Chaque composant ne prend **que ce qu'il utilise** : `Connexion` prend `connecter`, `Compte` prend `utilisateur`.
> - `{utilisateur.nom}` lit la propriété `nom` de l'objet. Ça ne marche que si `utilisateur` n'est pas `null` : c'est `RouteProtegee` qui s'en assure.

Avec le bouchon : la page s'affiche sans erreur, et « Mon compte » renvoie à la connexion. Se connecter ne fait rien : c'est le bouchon. La vérification qui suit se fait **ensemble, à la recette**.



BIB-2, avec le bouchon : personne n'est connecté, « Mon compte » renvoie ici

Vérification : tout marche comme à la fin de la séance 2. Connectez-vous (`lea.martin@exemple.fr`, `lecture`) : dans *Components*, l'état d'`AuthProvider` contient Léa. Déconnectez-vous : `null`.

Fin du sprint 1 · livrer, relire, fusionner · ensemble · 15 min

1. **Chacun livre son ticket.** A :

```
git status
git add .
git commit -m "BIB-1 : le contexte de connexion"
git push -u origin bib-1-contexte-connexion
```

Et B :

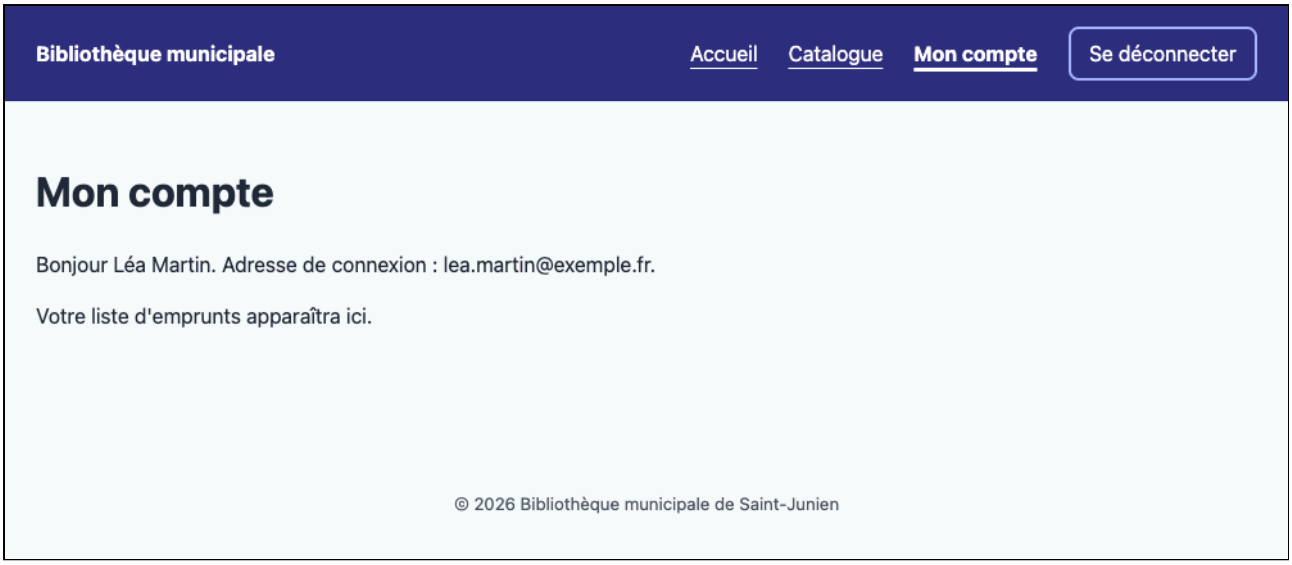
```
git status
git add .
git commit -m "BIB-2 : les composants lisent la connexion"
git push -u origin bib-2-lire-la-connexion
```

1. **Chacun ouvre sa pull request** sur GitHub : *Compare & pull request*, puis *Create pull request*.
2. **Chacun relit celle de l'autre**, onglet *Files changed*, et **l'auteur explique son code** en deux minutes. A : « que fait `AuthProvider`, et pourquoi `useAuth` lève-t-il une erreur ? » B : « qu'est-ce qui a disparu d'`App`, et où est passé `utilisateur` ? » Quand c'est compris : *Review changes → Approve*.
3. **Chacun fusionne la sienne** : *Merge pull request → Confirm merge → Delete branch*.
4. **Tous les deux** récupèrent le code réuni :

```
git switch main
git pull
```

1. **La recette.** Chez vous deux, faites la vérification de l'étape 3 : la connexion marche, et l'état d'`AuthProvider` change dans *Components*. Le bouchon a disparu, sans que personne n'ait copié un fichier.
2. **Ça ne marche pas ?** Comparez les noms du contrat, lettre par lettre : `utilisateur`, `connecter`, `deconnecter`. C'est presque toujours là.

> **Git derrière**
>
> - Vos deux branches ont modifié **des fichiers différents** : Git les réunit sans rien vous demander. C'est pour ça que les tickets disent quels fichiers ils touchent.
> - Si deux personnes modifient **la même ligne**, Git ne choisit pas à leur place : il signale un **conflit**. Le découpage des tickets vous l'évite aujourd'hui.



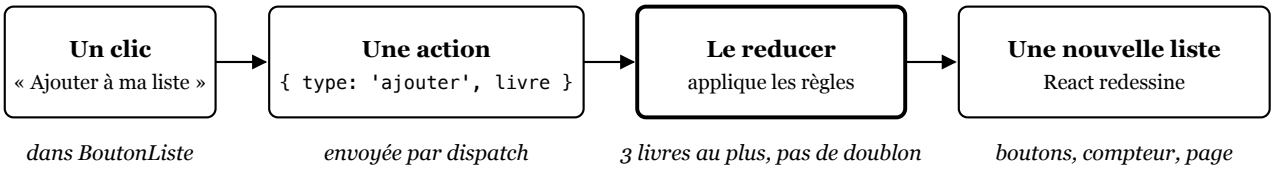
Fin du sprint 1 : Léa est connectée, le menu propose « Se déconnecter »

À vous

S'il vous reste du temps, à deux : l'un écrit sur une nouvelle branche, l'autre relit la pull request. Affichez le **prénom** de l'adhérent connecté dans le pied de page (« Connecté·e : Léa »), **sans toucher à `App`** ni à aucune prop.

Partie 2 · Sprint 2 · Un reducer pour la liste

Les rôles s'inversent. BIB-3, pour B : le reducer et son contexte (étapes 4 et 5). **BIB-4, pour A** : ce qui s'en sert, le bouton, le compteur et la page « Ma liste » (étapes 6, 7 et 8). Vous travaillez en même temps. BIB-5 viendra après la fusion.



Ce que fait le reducer du sprint 2, à chaque clic

Le contrat du sprint 2, à lire ensemble avant de commencer :

Fichier	Ce qu'il fournit	Ce que c'est
`src/contexts/listeReducer.js`	`MAXIMUM`	Le nombre de livres autorisés : `3`
	`listeReducer`	Le reducer : les règles de la liste
`src/contexts/ListeContext.jsx`	`ListeProvider`	Le composant à placer autour de l'application
	`useListe()`	Une fonction qui renvoie un objet avec six choses :
	→ `liste`	Un tableau de livres `{ id, titre, auteur }`
	→ `pleine`	Un booléen : la liste a-t-elle ses trois livres ?
	→ `contient(id)`	Une fonction : ce livre est-il dans la liste ?
	→ `ajouter(livre)`, `retirer(id)`, `vider()`	Les trois fonctions qui changent la liste

Ces deux fichiers existent déjà : ce sont des bouchons. Ouvrez-les ensemble. La liste y contient toujours *Clean Code*, et les trois fonctions ne font rien.

Chacun crée sa branche, à partir d'un `main` à jour. B :

```
git switch main
git pull
git switch -c bib-3-reducer-liste
```

Et A :

```
git switch main
git pull
git switch -c bib-4-bouton-compteur-page
```

Étape 4 · B · BIB-3 · Le reducer : toutes les règles au même endroit · 15 min

La liste d'emprunts change de trois façons : **ajouter** un livre, en **retirer** un, tout **vider**. Et elle a des règles : pas deux fois le même livre, **trois au plus**. Plutôt que d'éparpiller ces règles dans les boutons, on les écrit **une fois**, dans une fonction : un **reducer**.

Remplacez tout le contenu de `src/contexts/listeReducer.js` par :

```
// La liste d'emprunts : au plus 3 livres, chacun une seule fois.
export const MAXIMUM = 3;

// Un reducer : (état actuel, action) → nouvel état. Il ne modifie rien, il ne fait
rien d'autre.
export function listeReducer(liste, action) {
  switch (action.type) {
    case 'ajouter': {
      const dejaLa = liste.some((l) => l.id === action.livre.id);
      if (dejaLa || liste.length >= MAXIMUM) {
        return liste;
      }
      const { id, titre, auteur } = action.livre;
      return [...liste, { id, titre, auteur }];
    }
    case 'retirer':
      return liste.filter((l) => l.id !== action.id);
    case 'vider':
      return [];
    default:
      throw new Error(`Action inconnue : ${action.type}`);
  }
}
```

Ce qu'on voit	**Pourquoi**
`(liste, action) => nouvelle liste`	Un reducer reçoit l'état actuel et **une action** (un objet avec un `type`), et **renvoie** le nouvel état
`[...liste, …]`, `filter`, `[]`	**Jamais** `push` ni `splice` : un **nouveau** tableau à chaque fois. C'est comme ça que React voit le changement
`return liste`	Rien à faire (déjà là, ou liste pleine) : on renvoie **le même** tableau, React ne redessine rien
`{ id, titre, auteur }`	On ne garde que ce que la page « Ma liste » affiche
Pas de React, pas de `localStorage`	Une fonction **pure** : mêmes entrées, même résultat, aucun effet de bord. Elle se teste toute seule

Le JavaScript derrière

- `switch (action.type) { case 'ajouter': … }` : un **aiguillage**. JavaScript compare `action.type` à chaque `case`, et exécute le premier qui correspond. `default` : aucun ne correspond.
- Le `return` d'un `case` sort de la fonction tout de suite : pas besoin de `break`.
- `liste.some((l) => …)` répond `true` si **au moins un** élément vérifie la condition. `filter` garde ceux qui la vérifient.
- `const { id, titre, auteur } = action.livre;` sort trois propriétés du livre ; `{ id, titre, auteur }` fabrique ensuite un nouvel objet avec seulement celles-là.
- Dans la console, `await import('…')` charge le fichier à la demande, et `await` attend qu'il soit chargé.

Vérification : le reducer se teste **sans React**. Dans la console du navigateur (l'application ouverte), tapez :

```
const { listeReducer } = await import('/src/contexts/listeReducer.js');
const dune = { id: 'dune', titre: 'Dune', auteur: 'Frank Herbert' };
const une = listeReducer([], { type: 'ajouter', livre: dune });
listeReducer(une, { type: 'ajouter', livre: dune }) === une;
```

La dernière ligne affiche `true` : le deuxième ajout du même livre n'a rien changé.

Étape 5 · B · BIB-3 · Le contexte de la liste · 10 min

Le reducer dit **comment** la liste change. `useReducer` le branche sur React : il garde la liste, et fournit **`dispatch`**, la fonction pour **envoyer une action**.

Remplacez tout le contenu de `src/contexts/ListeContext.jsx` par :

```
import { createContext, useContext, useReducer } from 'react';
import { listeReducer, MAXIMUM } from './listeReducer.js';

const ListeContext = createContext(null);

export function ListeProvider({ children }) {
  const [liste, dispatch] = useReducer(listeReducer, []);

  const valeur = {
    liste,
    pleine: liste.length >= MAXIMUM,
    contient: (id) => liste.some((l) => l.id === id),
    ajouter: (livre) => dispatch({ type: 'ajouter', livre }),
    retirer: (id) => dispatch({ type: 'retirer', id }),
    vider: () => dispatch({ type: 'vider' }),
  };

  return <ListeContext value={valeur}>{children}</ListeContext>;
}

export function useListe() {
  const contexte = useContext(ListeContext);
  if (!contexte) {
    throw new Error('useListe doit être utilisé dans un <ListeProvider>.');
  }
  return contexte;
}
```

Dans `src/main.jsx`, juste après la ligne `import { AuthProvider } from './contexts/AuthContext.jsx';` :

```
import { ListeProvider } from './contexts/ListeContext.jsx';
```

Dans `src/main.jsx`, remplacez :

```
        <App />
```

par :

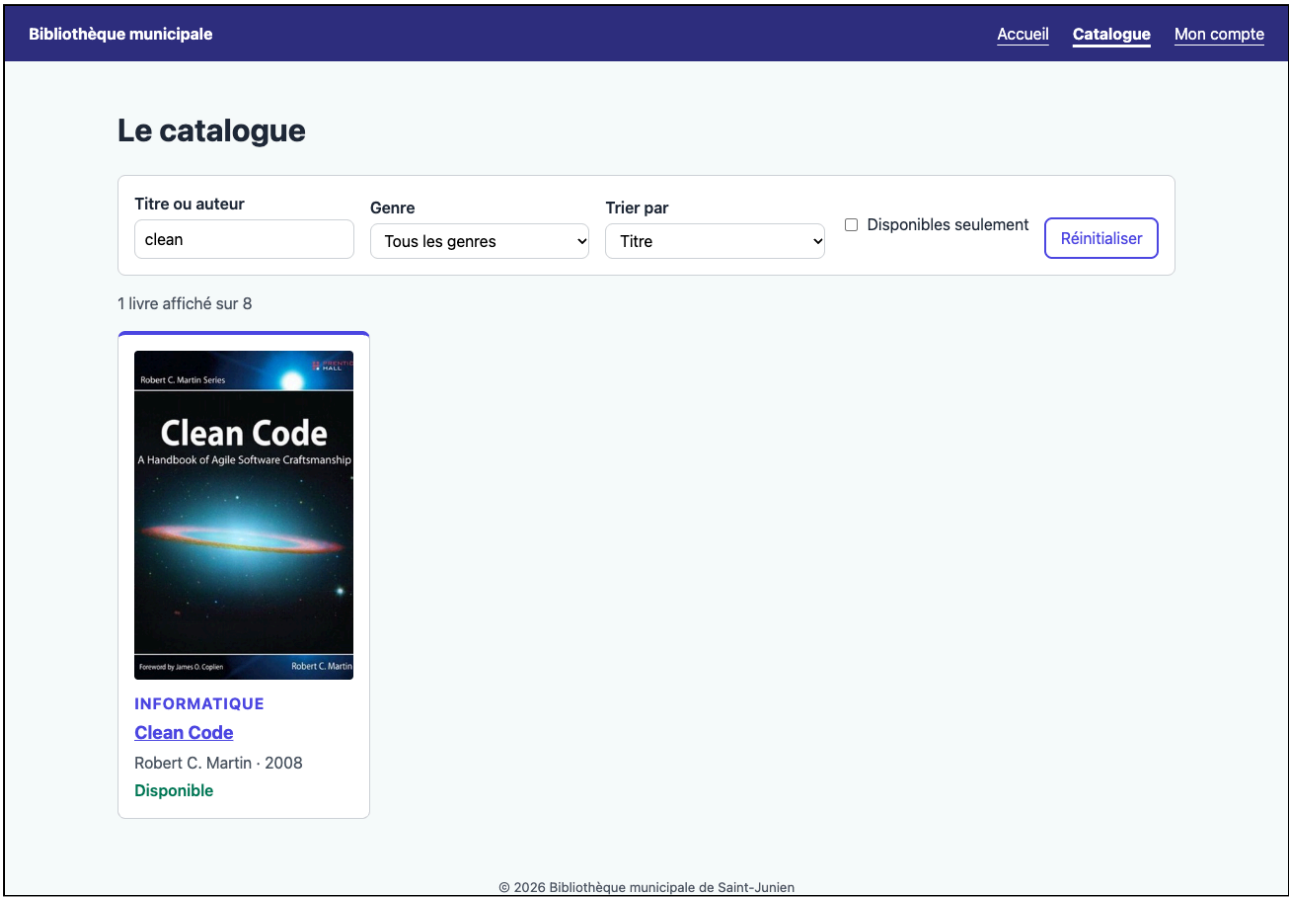
```
        <ListeProvider>
          <App />
        </ListeProvider>
```

On écrit	Ce que ça fait
`useReducer(listeReducer, [])`	Comme `useState([])`, mais les changements passent par le reducer
`dispatch({ type: 'ajouter', livre })`	« Voici ce qui s'est passé » : React appelle `listeReducer(liste, action)`, garde le résultat, redessine
`ajouter`, `retirer`, `vider`	Les composants n'écrivent pas d'actions à la main : ils appellent ces fonctions. Le **format** des actions reste une affaire du contexte
`pleine`, `contient`	Des valeurs **calculées** à partir de la liste, pour que les boutons n'aient pas à refaire le calcul

Le JavaScript derrière

- `const [liste, dispatch] = useReducer(listeReducer, [])` : comme `useState`, `useReducer` renvoie deux choses, rangées par **déstructuration de tableau** : la valeur, et la fonction pour la changer.
- `{ type: 'ajouter', livre }` est le raccourci de `{ type: 'ajouter', livre: livre }`.
- `contient: (id) => liste.some(…)` : une **fonction rangée dans un objet**. On l'appelle comme une fonction : `contient('dune')`.
- `pleine: liste.length >= MAXIMUM` : une comparaison donne un booléen, `true` ou `false`, rangé directement dans l'objet.

Vérification : dans *Components*, un `ListeProvider` sous l'`AuthProvider`, avec un état `Reducer : []`.



BIB-3 : à l'écran, le catalogue n'a pas encore de bouton

Étape 6 · A · BIB-4 · Un bouton « Ajouter à ma liste » · 15 min

A, vous commencez en même temps que B, sur votre branche. Vous utilisez les deux bouchons : la liste contient toujours *Clean Code*, et les clics ne changent rien, c'est normal. **Aux étapes 6, 7 et 8**, vous devez voir : *Clean Code* « Dans ma liste », le compteur du menu à 1, et *Clean Code* dans la page « Ma liste ». Les vérifications écrites dans la fiche décrivent le vrai comportement : vous les ferez **ensemble, à la recette**.

Le même bouton ira sur les cartes et sur la fiche : **un composant**, qui lit la liste lui-même.

Créez `src/components/BoutonListe.jsx` :

```
import { useListe } from '../contexts/ListeContext.jsx';

// Le même bouton sur les cartes et sur la fiche : il sait tout seul dans quel état
il est.
export default function BoutonListe({ livre }) {
  const { contient, pleine, ajouter } = useListe();

  if (!livre.disponible) {
    return <button type="button" disabled>Indisponible</button>;
  }
  if (contient(livre.id)) {
    return <button type="button" disabled>Dans ma liste</button>;
  }
  if (pleine) {
    return <button type="button" disabled>Liste pleine</button>;
  }
  return (
    <button type="button" onClick={() => ajouter(livre)}>
      Ajouter à ma liste
    </button>
  );
}
```

Dans `src/components/CarteLivre.jsx`, juste après la ligne `import { Link } from 'react-router';` :

```
import BoutonListe from './BoutonListe.jsx';
```

Dans `src/components/CarteLivre.jsx`, juste après la ligne `<p className="dispo">{livre.disponible ? 'Disponible' : 'Emprunté'}</p>` :

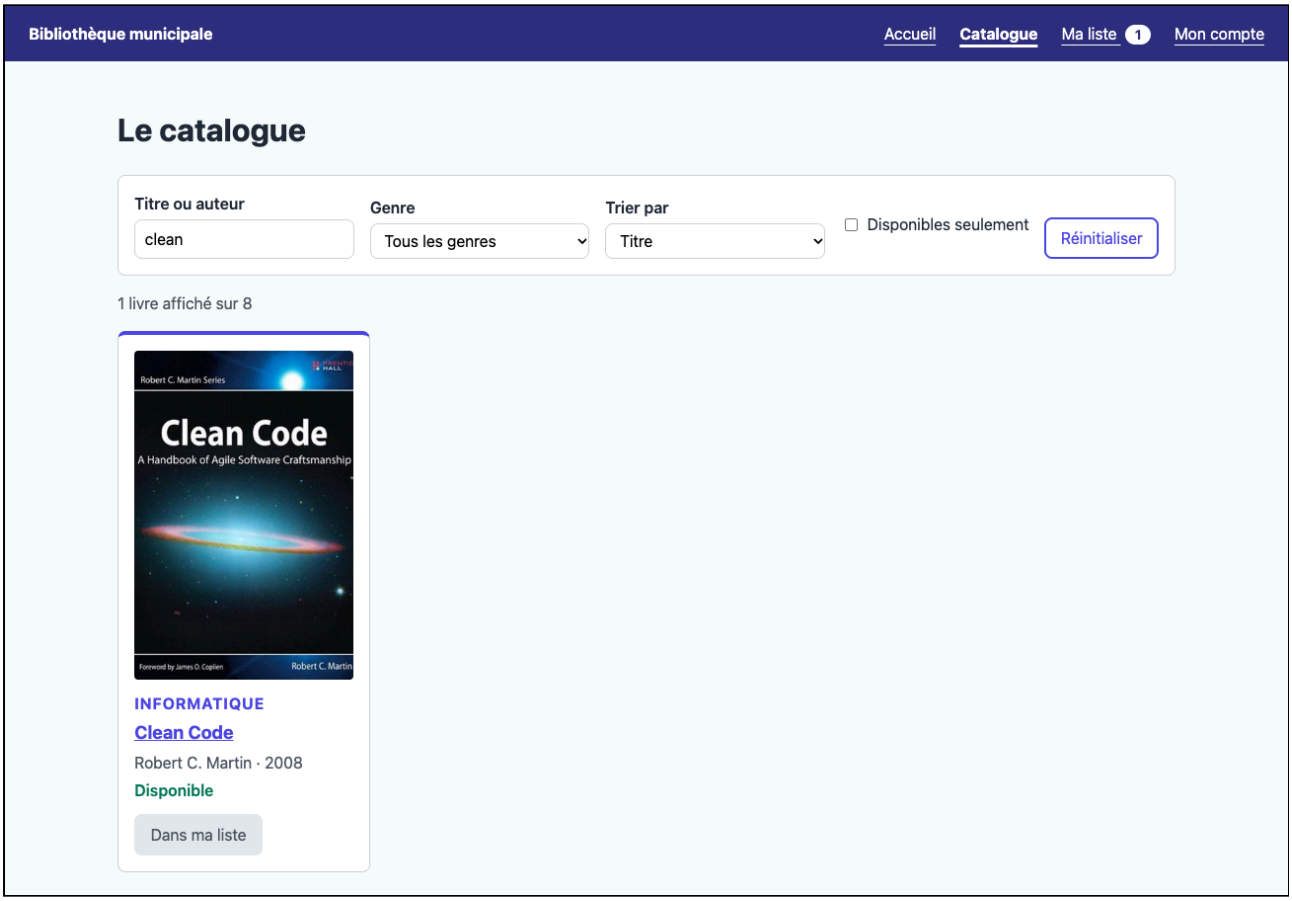
```
      <div className="actions">
        <BoutonListe livre={livre} />
      </div>
```

`CarteLivre` ne reçoit toujours qu'un livre. Elle ne sait rien de la liste ; `BoutonListe` va la chercher dans le contexte. Ni `Catalogue`, ni `ListeLivres` n'ont changé : c'est tout l'intérêt.

Le JavaScript derrière

- Trois `if` qui font chacun un `return` : la fonction **s'arrête au premier** qui convient. L'ordre des tests est donc une règle : un livre indisponible reste « Indisponible », même s'il était dans la liste.
- `!livre.disponible` : `!` inverse un booléen. « Si le livre n'est **pas** disponible ».
- `onClick={() => ajouter(livre)}` : on donne une fonction **à appeler plus tard**, au clic. `onClick={ajouter(livre)}` l'appellerait tout de suite, à chaque affichage.
- `disabled` écrit seul, sans valeur, vaut `disabled={true}`.

Vérification : dans le catalogue, « Ajouter à ma liste » sur *Dune*… non : *Dune* est « Indisponible ». Sur *Clean Code* : le bouton devient « Dans ma liste ». Dans *Components*, l'état du `ListeProvider` contient *Clean Code*.



BIB-4, sur la branche de A, avec les bouchons : Clean Code est toujours « Dans ma liste », le compteur dit 1

Étape 7 · A · BIB-4 · Le compteur dans le menu · 5 min

Dans `src/components/MiseEnPage.jsx`, juste après la ligne `import { useAuth } from '../contexts/AuthContext.jsx';` :

```
import { useListe } from '../contexts/ListeContext.jsx';
```

Dans `src/components/MiseEnPage.jsx`, juste après la ligne `const { utilisateur, deconnecter } = useAuth();` :

```
  const { liste } = useListe();
```

Dans `src/components/MiseEnPage.jsx`, juste après la ligne `<li><NavLink to="/catalogue">Catalogue</NavLink></li>` :

```
            <li>
              <NavLink to="/ma-liste">
                Ma liste <span className="pastille">{liste.length}</span>
              </NavLink>
            </li>
```

> **Le JavaScript derrière**
>
> - `{liste.length}` : le nombre d'éléments du tableau. La liste vient du contexte : le menu se redessine **tout seul** dès qu'elle change, sans qu'aucune prop ne le prévienne.

Vérification : ajoutez deux livres : le compteur du menu passe à 2, **instantanément**. Deux composants que rien ne relie dans l'arbre (le menu et une carte) partagent la même donnée.

Étape 8 · A · BIB-4 · La page « Ma liste » · 15 min

Créez `src/pages/MaListe.jsx` :

```
import { Link } from 'react-router';
import { useListe } from '../contexts/ListeContext.jsx';
import { MAXIMUM } from '../contexts/listeReducer.js';

export default function MaListe() {
  const { liste, retirer, vider } = useListe();
  const s = liste.length > 1 ? 's' : '';

  function confirmerVider() {
    if (window.confirm('Retirer tous les livres de votre liste ?')) {
      vider();
    }
  }

  return (
    <>
      <title>Ma liste — Bibliothèque municipale</title>
      <h1>Ma liste d'emprunts</h1>

      {liste.length === 0 ? (
        <p className="aucun">
          Votre liste est vide. <Link to="/catalogue">Choisir des livres dans le
catalogue</Link>
        </p>
      ) : (
        <>
          <p className="compteur">{liste.length} livre{s} sur {MAXIMUM} possibles.
</p>
          <ul className="selection">
            {liste.map((livre) => (
              <li key={livre.id}>
                <p><Link to={`/livres/${livre.id}`}>{livre.titre}</Link>,
{livre.auteur}</p>
                <button
                  type="button"
                  className="secondaire"
                  aria-label={`Retirer ${livre.titre}`}
```

```
                onClick={() => retirer(livre.id)}
              >
                Retirer
              </button>
            </li>
          ))}
        </ul>
        <p className="barre">
          <button type="button" className="secondaire" onClick={confirmerVider}>
            Vider la liste
          </button>
        </p>
      </>
    )}
  </>
 );
}
```

Dans `src/App.jsx`, juste après la ligne `import FicheLivre from './pages/FicheLivre.jsx';` :

```
import MaListe from './pages/MaListe.jsx';
```

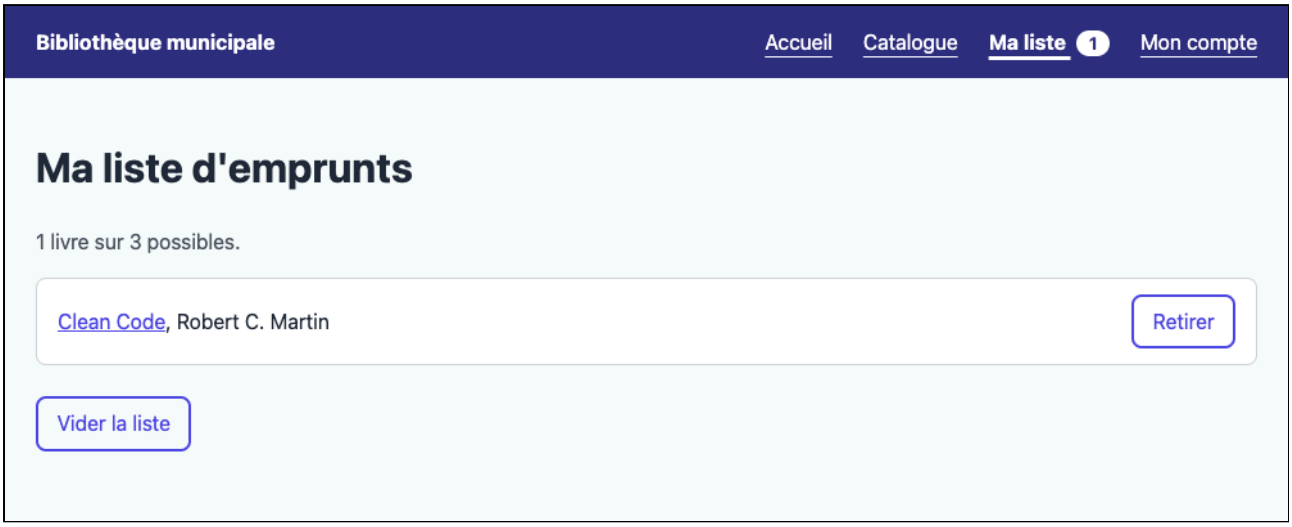
Dans `src/App.jsx`, juste après la ligne `<Route path="livres/:id" element={<FicheLivre />} />` :

```
        <Route path="ma-liste" element={<MaListe />} />
```

`aria-label={Retirer ${livre.titre}}` : trois boutons « Retirer » identiques ne disent pas, à un lecteur d'écran, **quoi** ils retirent. L'étiquette complète le dit ; le texte visible reste court.

> **Le JavaScript derrière**
>
> - `const s = liste.length > 1 ? 's' : '';` : la condition ternaire choisit le pluriel ; `livre{s}` écrit « livre » ou « livres ».
> - `{liste.length === 0 ? (…) : (…)}` : selon que la liste est vide ou non, React affiche l'un **ou** l'autre bloc. `===` compare strictement.
> - `window.confirm('…')` ouvre la boîte « OK / Annuler » du navigateur, et renvoie `true` si l'on clique OK : le `if` ne vide la liste que dans ce cas.
> - `confirmerVider` est déclarée **dans** le composant : elle voit `vider`, sorti de `useListe()` juste au-dessus.

Vérification : trois livres dans la liste, puis « Ma liste » : les trois, avec leur lien. Retirez-en un : il disparaît, le compteur du menu suit, et son bouton redevient « Ajouter » dans le catalogue. « Vider la liste », « OK » : la liste est vide, avec le lien vers le catalogue.



BIB-4, avec les bouchons : la page « Ma liste »

Fin du sprint 2 · livrer, relire, fusionner · ensemble · 15 min

1. **Chacun livre son ticket.** B :

```
git status
git add .
git commit -m "BIB-3 : le reducer et le contexte de la liste"
git push -u origin bib-3-reducer-liste
```

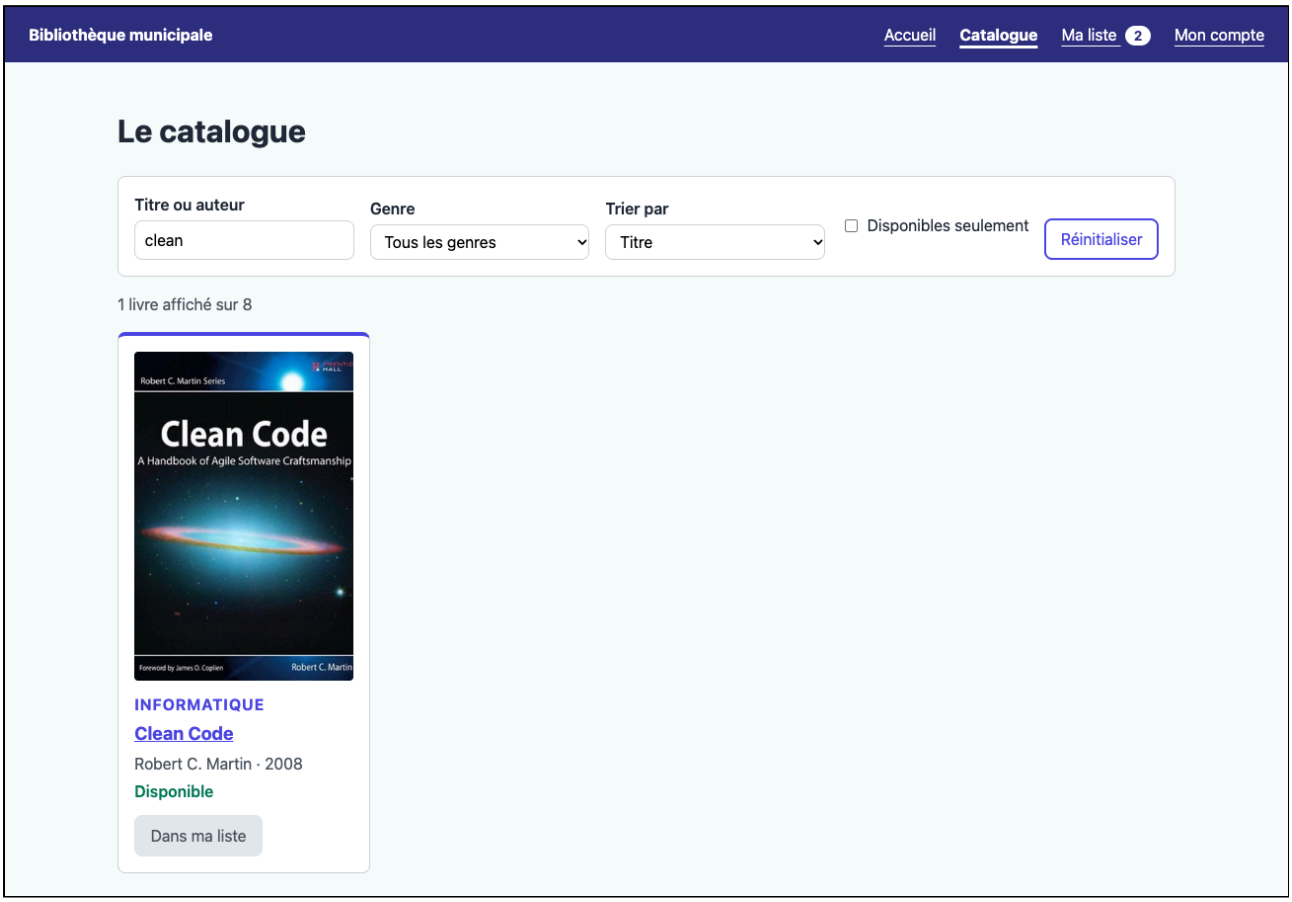
Et A :

```
git status
git add .
git commit -m "BIB-4 : le bouton, le compteur et la page Ma liste"
git push -u origin bib-4-bouton-compteur-page
```

1. **Chacun ouvre sa pull request**, **relit celle de l'autre**, et **l'auteur explique son code**. B : « que fait le reducer quand on ajoute un livre déjà présent ? » A : « comment `BoutonListe` sait-il quoi afficher, sans recevoir la liste en prop ? » Puis *Approve*.
2. **Chacun fusionne la sienne**, et **tous les deux** se remettent à jour :

```
git switch main
git pull
```

1. **La recette.** Chez vous deux, faites les vérifications des étapes 6, 7 et 8 : les boutons changent, le compteur suit, la page « Ma liste » retire et vide.
2. **Ensuite, chacun reprend son clavier** : B fait BIB-5 (l'étape 9). Pendant ce temps, A lit les deux fichiers de B et fait l'« À vous » de la console, juste en dessous : au sprint 3, c'est A qui les modifiera.



Fin du sprint 2 : deux livres ajoutés, le compteur dit 2

À vous · A, dans la console

Pendant que B fait l'étape 9. C'est en testant le reducer de B que vous le comprendrez.

Dans la console du navigateur, chargez d'abord le reducer :

```
const { listeReducer } = await import('/src/contexts/listeReducer.js');
```

Puis vérifiez qu'un **quatrième** livre n'entre pas : fabriquez une liste de trois livres avec trois actions `{ type: 'ajouter', livre: … }`, et essayez d'en ajouter un autre.

À vous · ensemble

À faire ensemble, à la recette. Ajoutez trois livres disponibles : les boutons des autres livres disponibles disent « Liste pleine ». Où est écrite la règle des trois livres ? Combien de fichiers faudrait-il modifier pour passer à cinq ?

Étape 9 · B · BIB-5 · Le même bouton sur la fiche · 5 min

B, après la fusion de BIB-3 et de BIB-4 : vous utilisez le bouton écrit par A. C'est pour ça qu'il fallait `git pull` avant.

Une nouvelle branche, à partir d'un `main` à jour :

```
git switch main
git pull
git switch -c bib-5-bouton-fiche
```

Dans `src/pages/FicheLivre.jsx`, juste après la ligne `import { livres } from '../data/livres.js';` :

```
import BoutonListe from '../components/BoutonListe.jsx';
```

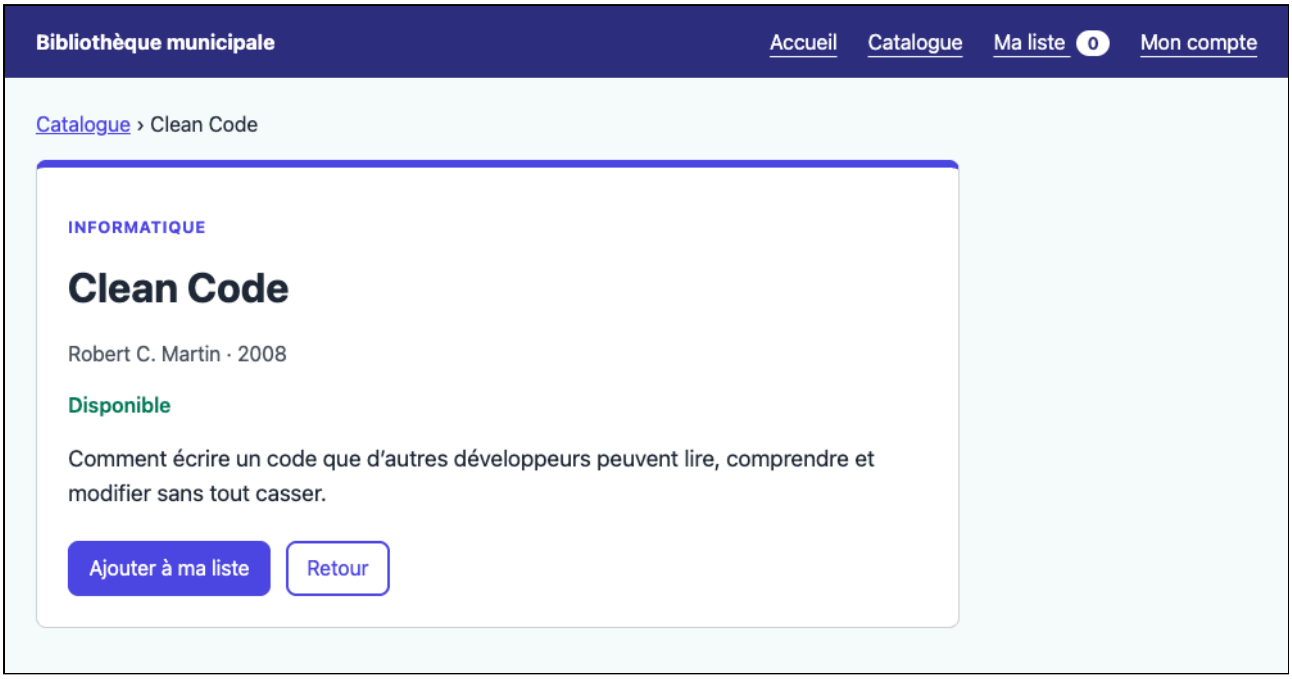
Dans `src/pages/FicheLivre.jsx`, juste après la ligne `<div className="actions">` :

```
          <BoutonListe livre={livre} />
```

Deux lignes. Le bouton sait déjà tout : si le livre est disponible, s'il est dans la liste, si elle est pleine. C'est le critère **DRY** de l'épreuve E2 : un composant écrit une fois, utilisé partout.

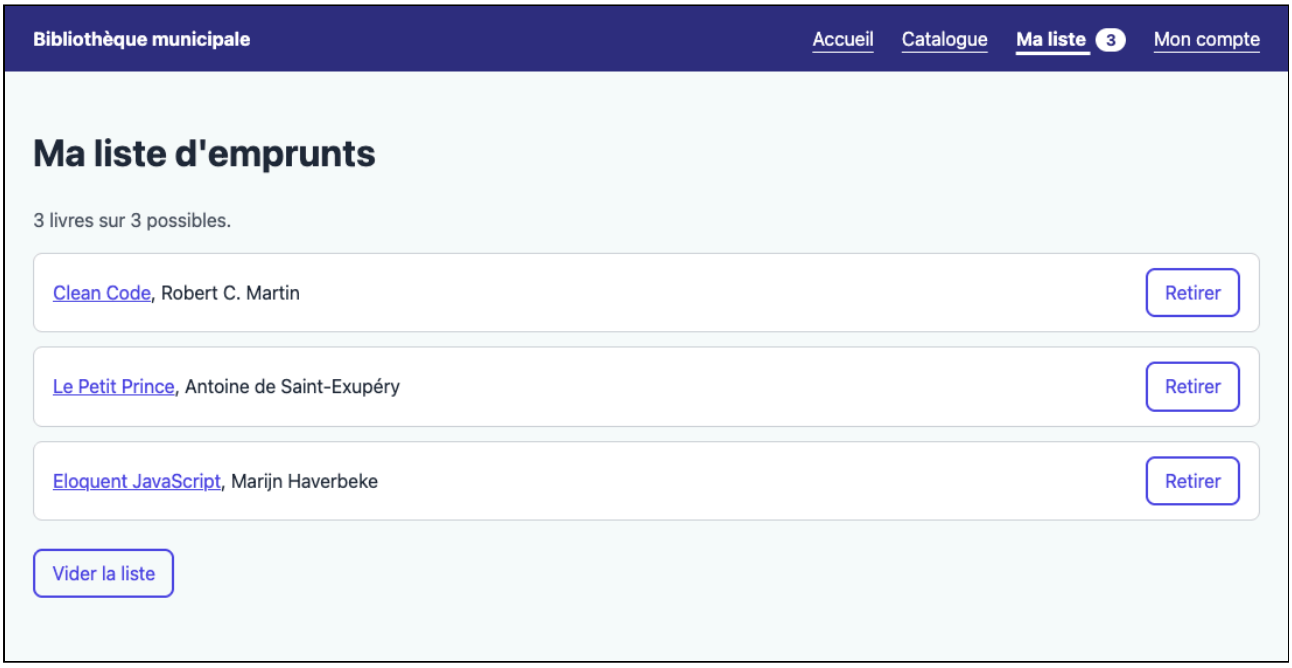
> **Le JavaScript derrière**
>
> - Un `import`, puis une balise : un composant se réutilise comme une fonction qu'on appelle à deux endroits. Son code n'existe qu'une fois.

Vérification : sur la fiche de *Clean Code*, le bouton est dans le même état que sur sa carte. Ajoutez depuis la fiche : le compteur du menu suit.



BIB-5 : le bouton sur la fiche d'un livre

Livrez BIB-5 comme les autres : `git add .`, `git commit -m "BIB-5 : le bouton sur la fiche"`, `git push -u origin bib-5-bouton-fiche`, une pull request, **relue par A**, fusionnée. Puis `git switch main` et `git pull`, tous les deux.



Fin de l'étape 9 : la page « Ma liste », avec trois livres

Partie 3 · Sprint 3 · Une mémoire

BIB-6, pour A : la mémoire, dans le contexte écrit par B (étape 10). **BIB-7, pour B** : la page du compte lit deux contextes (étape 11). Pas de bouchon ni de contrat cette fois : vous modifiez deux fichiers différents, et rien ne change dans ce que `useListe()` renvoie.

Ce sprint peut se faire à distance : vous n'avez pas besoin d'être dans la même pièce, puisque tout passe par GitHub.

Chacun crée sa branche, à partir d'un `main` à jour (BIB-5 doit être fusionné). A :

```
git switch main
git pull
git switch -c bib-6-memoire
```

Et B :

```
git switch main
git pull
git switch -c bib-7-compte
```

Étape 10 · A · BIB-6 · Survivre au rechargement · 15 min

Faites l'essai : trois livres dans la liste, puis `F5`. La liste est vide. L'état de React vit dans la mémoire de l'onglet : un rechargement l'efface.

`localStorage` (vu dans la remise à niveau) garde des textes d'une visite à l'autre. Deux moments : **lire** la liste au démarrage, **l'écrire** à chaque changement.

Dans `src/contexts/ListeContext.jsx`, remplacez :

```
import { createContext, useContext, useReducer } from 'react';
```

par :

```
import { createContext, useContext, useEffect, useReducer } from 'react';
```

Dans `src/contexts/ListeContext.jsx`, juste après la ligne `const ListeContext = createContext(null);` :

```
const CLE = 'liste-emprunts';

// Au premier affichage, on relit la liste enregistrée ; si elle est absente ou abîmée, liste vide.
function charger() {
  try {
    return JSON.parse(localStorage.getItem(CLE)) ?? [];
  } catch {
    return [];
  }
}
```

Dans `src/contexts/ListeContext.jsx`, remplacez :

```
  const [liste, dispatch] = useReducer(listeReducer, []);
```

par :

```
  const [liste, dispatch] = useReducer(listeReducer, [], charger);

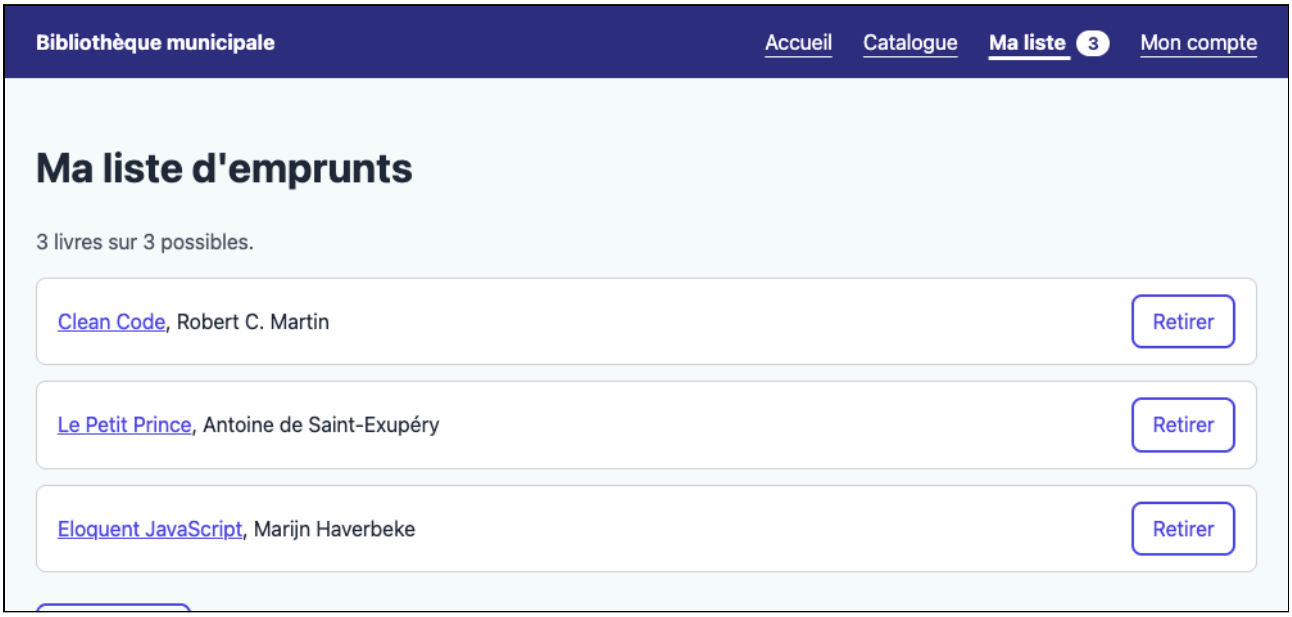
  // Après chaque changement de la liste, on l'enregistre.
  useEffect(() => {
    localStorage.setItem(CLE, JSON.stringify(liste));
  }, [liste]);
```

On écrit	Ce que ça fait
`useReducer(listeReducer, [], charger)`	Le **troisième** argument est une **fonction d'initialisation** : React l'appelle **une seule fois**, au premier affichage, avec le deuxième argument (`[]`), et prend ce qu'elle renvoie comme liste de départ
`charger`, pas `charger()`	On **donne** la fonction : avec les parenthèses, `localStorage` serait relu à **chaque** rendu, pour rien
`try … catch`	Si quelqu'un a abîmé la valeur enregistrée, `JSON.parse` lève une erreur : la page ne doit pas planter pour ça
`useEffect(…, [liste])`	Un **effet** : du code qui agit **en dehors** de React (ici, le stockage du navigateur), exécuté **après** l'affichage, et seulement quand `liste` a changé

Pourquoi pas dans le reducer ? Il doit rester **pur** : il calcule, il n'écrit nulle part. Écrire dans `localStorage` est un **effet de bord** : sa place est dans un effet.

> **Le JavaScript derrière**
>
> - `localStorage` ne garde que du **texte**. `JSON.stringify(liste)` transforme le tableau en texte ; `JSON.parse(texte)` refait le tableau.
> - `localStorage.getItem(CLE)` renvoie `null` si rien n'est enregistré ; `JSON.parse(null)` donne `null`, et `?? []` le remplace par une liste vide.
> - `try { … } catch { … }` : on **essaie** un code ; s'il lève une erreur, on passe dans le `catch` au lieu de planter.
> - `useEffect(() => { … }, [liste])` : deux arguments, la fonction à exécuter et le **tableau des dépendances**. React la relance chaque fois que `liste` change.

Vérification : trois livres, `F5` : ils sont toujours là, et le compteur aussi. Onglet *Application* (Chrome) ou *Stockage* (Firefox) → *Local Storage* → `http://localhost:5173` : la clé `liste-emprunts`, et son texte.



BIB-6 : après un rechargement, la liste est intacte

Étape 11 · B · BIB-7 · Le compte connaît la liste · 10 min

Une page peut lire **plusieurs** contextes.

Remplacez tout le contenu de `src/pages/Compte.jsx` par :

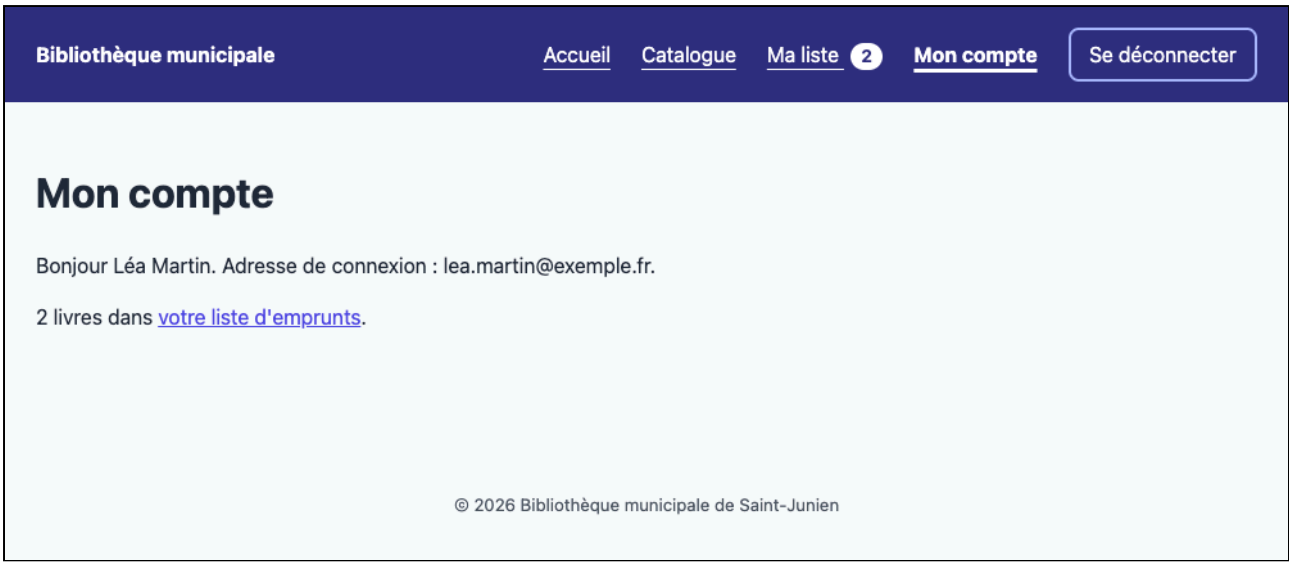
```
import { Link } from 'react-router';
import { useAuth } from '../contexts/AuthContext.jsx';
import { useListe } from '../contexts/ListeContext.jsx';

export default function Compte() {
  const { utilisateur } = useAuth();
  const { liste } = useListe();
  const s = liste.length > 1 ? 's' : '';

  return (
    <>
      <title>Mon compte — Bibliothèque municipale</title>
      <h1>Mon compte</h1>
      <p>Bonjour {utilisateur.nom}. Adresse de connexion : {utilisateur.email}.</p>
      <p>
        {liste.length} livre{s} dans <Link to="/ma-liste">votre liste
d'emprunts</Link>.
      </p>
    </>
  );
}
```

> **Le JavaScript derrière**
>
> - Deux hooks à la suite, `useAuth()` puis `useListe()` : un composant lit autant de contextes qu'il en a besoin. Chacun renvoie son objet, dont on sort ce qui sert.

Vérification : connecté, deux livres dans la liste : « 2 livres dans votre liste d'emprunts ».



BIB-7 : le compte rappelle le nombre de livres

Fin du sprint 3 · livrer, relire, fusionner · ensemble · 10 min

1. **Chacun livre son ticket** : `git status`, `git add .`, puis `git commit -m "BIB-6 : la liste survit au rechargement"` et `git push -u origin bib-6-memoire` pour A ; `git commit -m "BIB-7 : le compte affiche la liste"` et `git push -u origin bib-7-compte` pour B.
2. **Pull requests, relecture, explication.** A explique à B ce qu'il a ajouté dans **son** fichier : que fait `charger`, et quand `useEffect` s'exécute-t-il ? Puis *Approve*, et chacun fusionne la sienne.
3. **Tous les deux** : `git switch main`, puis `git pull`.
4. **La recette.** Chez vous deux : la liste survit à `F5`, et le compte l'affiche.

Étape 12 · Regarder l'état · 10 min

Ensemble, sur un seul écran.

Dans l'onglet *Components* :

1. Cliquez sur `ListeProvider` : à droite, le *Reducer* et sa valeur, la liste.
2. Ajoutez un livre : la valeur change ; les composants qui lisent la liste **clignotent** (si *Highlight updates* est coché, séance 1).
3. Cliquez sur une `CarteLivre`, puis sur son `BoutonListe` : à droite, *Context* montre ce qu'il lit.

Vérification : vous savez montrer, dans l'arbre, **qui détient** la liste (un seul composant) et **qui la lit** (le menu, les boutons, la page « Ma liste », le compte).

À vous

À discuter à deux : mettez-vous d'accord sur une réponse. Rechargez la page en étant connecté : la liste est restée, **mais pas la connexion**. Pourquoi ? Faut-il la garder aussi dans `localStorage` ? (Réponse argumentée, pas de code : la séance 4 le fera avec un vrai jeton.)

Pour finir

Contrôle final

- [ ] « Ajouter à ma liste » sur les cartes **et** sur la fiche ; « Indisponible », « Dans ma liste », « Liste pleine » quand il le faut
- [ ] Le compteur du menu suit chaque ajout, chaque retrait
- [ ] « Ma liste » : retirer un livre, tout vider (avec confirmation), le message quand elle est vide
- [ ] Jamais deux fois le même livre, jamais plus de trois
- [ ] La liste **survit au rechargement**
- [ ] `App` n'a **plus aucun état** et ne passe **plus aucune prop** : chaque composant lit les contextes dont il a besoin
- [ ] Aucune erreur dans la console
- [ ] Il ne reste **aucun bouchon** : cherchez le mot « BOUCHON » dans `src/`
- [ ] Sur GitHub : **huit pull requests fusionnées** (BIB-0 à BIB-7), chacune relue par l'autre
- [ ] `main` est à jour chez vous deux : `git status` dit « nothing to commit », et `git pull` « Already up to date »
- [ ] Chacun sait expliquer le code **de l'autre**

Rendre le TP

Le **vendredi 13 novembre à 18 h 00**. Sur la page de votre dépôt GitHub, branche `main` : le bouton vert *Code*, puis *Download ZIP*. Renommez le fichier en **`NOM1-NOM2-React-S3.zip`** (les deux noms du binôme), et déposez-le **une seule fois** dans **Rendus — à déposer ici → Frameworks React → FM04 · Séance 3**.

Ce zip ne contient pas `node_modules` : Git ne l'a jamais enregistré. Et votre README donne l'adresse du dépôt : je pourrai y lire vos commits et vos relectures.

Git : les commandes de la journée

Commande	Ce qu'elle fait
`git status`	Où en suis-je ? La branche, les fichiers modifiés
`git switch main`	Revenir sur la branche principale
`git pull`	Récupérer ce qui a changé sur GitHub
`git switch -c bib-N-nom`	Créer une branche, et s'y placer
`git add .`	Choisir tous les fichiers modifiés pour le prochain commit
`git commit -m "BIB-N : …"`	Enregistrer une étape, avec son message
`git push -u origin bib-N-nom`	Envoyer la branche sur GitHub
`git log --oneline`	Relire l'historique, une ligne par commit

Séance 4 · mardi 17 novembre, 8 h 30 : le catalogue parle à une vraie API. Vous repartirez de ce projet, ou du zip de départ de la séance 4.

Pour aller plus loin

- Le **nombre** de livres de la liste pourrait s'afficher dans l'onglet du navigateur : « (3) Catalogue — Bibliothèque ». Où le calculer, sans le répéter dans chaque page ?
- Ajoutez une action `'deplacer'` au reducer, pour changer l'ordre de la liste (monter, descendre). Testez-la d'abord dans la console, comme à l'étape 4.